

CO 2 ASSESSMENT TOOL – 3

DSA0134 – Object Oriented Programming with C++

NAME: T. Nivya Sri

REG NO: 192424131

1. Student Analytics System:

```
#include <iostream>

#include <string>

#include <iomanip>

using namespace std;

class Department {

private:

    string deptName;

    int numStudents;

    string studentNames[50];

    float marks[50];

    static int totalUniversityStudents;

public:

    void inputDetails() {

        cout << "Enter Department Name: ";

        cin >> deptName;

        cout << "Enter Number of Students: ";

        cin >> numStudents;

        for(int i = 0; i < numStudents; i++) {

            cout << "Enter Name and Marks for student " << i + 1 << ": ";

            cin >> studentNames[i] >> marks[i];

            totalUniversityStudents++;

        }

    }

    void processDepartment(string &overallTopper, float &highestOverallMarks) {
```

```

if (numStudents == 0) {
    cout << "--- " << deptName << " Stats ---\nNo students in this department.\n";
    return;
}
float deptHighest = -1;
string deptTopper = "";
int passCount = 0;
for(int i = 0; i < numStudents; i++) {
    if (marks[i] >= 40.0) passCount++;
    if (marks[i] > deptHighest) {
        deptHighest = marks[i];
        deptTopper = studentNames[i];
    }
}
if (deptHighest > highestOverallMarks) {
    highestOverallMarks = deptHighest;
    overallTopper = deptTopper;
}
float passPercentage = ((float)passCount / numStudents) * 100;
cout << "--- " << deptName << " Stats ---\n";
cout << "Department Topper: " << deptTopper << " (" << deptHighest << " marks)\n";
cout << "Pass Percentage: " << fixed << setprecision(2) << passPercentage << "%\n";
}
static void displayTotalStudents() {
    cout << "\nTotal Students Across University: " << totalUniversityStudents << endl;
}
};
int Department::totalUniversityStudents = 0;
int main() {
    int numDepts;

```

```

cout << "Enter number of departments: ";
cin >> numDepts;
Department depts[10];
string overallTopper = "None";
float highestOverallMarks = -1;
for(int i = 0; i < numDepts; i++) {
    cout << "\n--- Department " << i + 1 << " ---\n";
    depts[i].inputDetails();
}
cout << "\n===== REPORTS =====\n";
for(int i = 0; i < numDepts; i++) {
    depts[i].processDepartment(overallTopper, highestOverallMarks);
}
Department::displayTotalStudents();
cout << "Overall University Topper: " << overallTopper
    << " (" << highestOverallMarks << " marks)\n";
return 0;
}

```

OUTPUT:

Output
<pre> Enter number of departments: 2 --- Department 1 --- Enter Department Name: science Enter Number of Students: 2 Enter Name and Marks for student 1: ram 78.2 Enter Name and Marks for student 2: ravi 84.2 --- Department 2 --- Enter Department Name: maths Enter Number of Students: 2 Enter Name and Marks for student 1: priya 92.8 Enter Name and Marks for student 2: praveen 89.9 ===== REPORTS ===== --- science Stats --- Department Topper: ravi (84.2 marks) Pass Percentage: 100.00% --- maths Stats --- Department Topper: priya (92.80 marks) Pass Percentage: 100.00% Total Students Across University: 4 Overall University Topper: priya (92.80 marks) </pre>

2. Election System

```
#include <iostream>

#include <string>

using namespace std;

class Region {
private:
    string regionName;
    int numCandidates;
    string candNames[20];
    int votes[20];
    static int totalCountryVotes;
public:
    void inputVotes() {
        cout << "Enter Region Name: ";
        cin >> regionName;
        cout << "Enter Number of Candidates: ";
        cin >> numCandidates;
        for(int i = 0; i < numCandidates; i++) {
            cout << "Enter Candidate Name and Votes: ";
            cin >> candNames[i] >> votes[i];
            totalCountryVotes += votes[i];
        }
    }

    void processRegion(string &overallWinner, int &highestOverallVotes, bool &overallTie)
    {
        if (numCandidates == 0) return;
        int maxVotes = -1;
        string regionWinner = "";
        bool regionTie = false;
        for(int i = 0; i < numCandidates; i++) {
            if (votes[i] > maxVotes) {
```

```

        maxVotes = votes[i];
        regionWinner = candNames[i];
        regionTie = false;
    } else if (votes[i] == maxVotes) {
        regionTie = true;
    }
}

if (maxVotes > highestOverallVotes) {
    highestOverallVotes = maxVotes;
    overallWinner = candNames[0]; // Temporary baseline
    overallTie = regionTie;
    if(!regionTie) overallWinner = regionWinner;
} else if (maxVotes == highestOverallVotes && maxVotes != -1) {
    overallTie = true;
}

cout << "Region: " << regionName << " | Winner: ";
if (regionTie) cout << "TIE";
else cout << regionWinner;
cout << " (" << maxVotes << " votes)\n";
}

static void displayTotalVotes() {
    cout << "\nTotal Votes Cast in Country: " << totalCountryVotes << endl;
}

};

int Region::totalCountryVotes = 0;

int main() {
    int numRegions;
    cout << "Enter number of regions: ";
    cin >> numRegions;
    Region regions[10];

```

```

string overallWinner = "None";
int highestOverallVotes = -1;
bool overallTie = false;
for(int i = 0; i < numRegions; i++) {
    cout << "\n--- Region " << i + 1 << " ---\n";
    regions[i].inputVotes();
}
cout << "\n===== ELECTION RESULTS =====\n";
for(int i = 0; i < numRegions; i++) {
    regions[i].processRegion(overallWinner, highestOverallVotes, overallTie);
}
Region::displayTotalVotes();
cout << "Overall Country Winner: ";
if (overallTie) cout << "TIE Situation!\n";
else cout << overallWinner << " with " << highestOverallVotes << " peak regional
votes.\n";
return 0;
}

```

OUTPUT:

```

Output
Enter number of regions: 2

--- Region 1 ---
Enter Region Name: north
Enter Number of Candidates: 2
Enter Candidate Name and Votes: rahul 5000
Enter Candidate Name and Votes: mohan 5000

--- Region 2 ---
Enter Region Name: south
Enter Number of Candidates: 2
Enter Candidate Name and Votes: ram 8948
Enter Candidate Name and Votes: gowtham 4321

===== ELECTION RESULTS =====
Region: north | Winner: TIE (5000 votes)
Region: south | Winner: ram (8948 votes)

Total Votes Cast in Country: 23269
Overall Country Winner: ram with 8948 peak regional votes.

=== Code Execution Successful ===

```

3. Retail Chain Management

```
#include <iostream>

#include <string>

using namespace std;

class Product {
private:
    string name;
    double price;
    int quantity;
public:
    void inputProduct() {
        cin >> name >> price >> quantity;
    }
    double getSales() const { return price * quantity; }
    string getName() const { return name; }
};

class Store {
private:
    string storeName;
    int numProducts;
    Product products[50];
    double storeTotal;
    static double chainTotalSales;
public:
    void inputStore() {
        cout << "Enter Store Name and Number of Products: ";
        cin >> storeName >> numProducts;
        storeTotal = 0;
        for(int i = 0; i < numProducts; i++) {
```

```

        cout << "Product " << i + 1 << " (Name, Price, Qty): ";
        products[i].inputProduct();
        storeTotal += products[i].getSales();
    }
    chainTotalSales += storeTotal;
}

double getStoreSales() const { return storeTotal; }
string getStoreName() const { return storeName; }
void findBestProduct(string &bestProdName, double &highestProdSales) {
    for(int i = 0; i < numProducts; i++) {
        if(products[i].getSales() > highestProdSales) {
            highestProdSales = products[i].getSales();
            bestProdName = products[i].getName();
        }
    }
}

static void displayChainTotal() {
    cout << "\nTotal Chain Sales: $" << chainTotalSales << endl;
}

};

double Store::chainTotalSales = 0;

int main() {
    int numStores;
    cout << "Enter number of stores: ";
    cin >> numStores;
    Store stores[10];
    string bestStore = "", bestProduct = "";
    double highestStoreSales = -1, highestProdSales = -1;
    for(int i = 0; i < numStores; i++) {
        cout << "\n--- Store " << i + 1 << " ---\n";
    }
}

```



```

    stores[i].inputStore();

    // Check for best store

    if (stores[i].getStoreSales() > highestStoreSales) {
        highestStoreSales = stores[i].getStoreSales();
        bestStore = stores[i].getStoreName();
    }

    // Check for best overall product

    stores[i].findBestProduct(bestProduct, highestProdSales);
}

cout << "\n===== RETAIL REPORT =====\n";

for(int i = 0; i < numStores; i++) {
    cout << "Store: " << stores[i].getStoreName() << " | Total Sales: $" <<
stores[i].getStoreSales() << endl;
}

Store::displayChainTotal();

cout << "Best Performing Store: " << bestStore << " ($" << highestStoreSales << ")\n";

cout << "Top Selling Product Overall: " << bestProduct << " ($" << highestProdSales <<
")\n";

return 0;
}

```

OUTPUT:

Output
<pre> Enter number of stores: 2 --- Store 1 --- Enter Store Name and Number of Products: Downtown 2 Product 1 (Name, Price, Qty): shoes 50 20 Product 2 (Name, Price, Qty): socks 5 10 --- Store 2 --- Enter Store Name and Number of Products: Uptown 1 Product 1 (Name, Price, Qty): jackets 120 5 ===== RETAIL REPORT ===== Store: Downtown Total Sales: \$1050 Store: Uptown Total Sales: \$600 Total Chain Sales: \$1650 Best Performing Store: Downtown (\$1050) Top Selling Product Overall: shoes (\$1000) === Code Execution Successful === </pre>

4. Sports Tournament Scoring System

```
#include <iostream>

#include <string>

using namespace std;

class Team {
private:
    string name;

    int matches, wins, draws, losses;

    int goalsScored, goalsConceded;

    int points, goalDiff;

    static int totalTeamsInTournament;
public:
    void inputTeam() {
        cout << "Enter Team Name, Wins, Draws, Losses, Goals Scored, Goals Conceded:\n";
        cin >> name >> wins >> draws >> losses >> goalsScored >> goalsConceded;

        matches = wins + draws + losses;

        points = (wins * 3) + (draws * 1);

        goalDiff = goalsScored - goalsConceded;

        totalTeamsInTournament++;
    }

    int getPoints() const { return points; }

    int getGoalDiff() const { return goalDiff; }

    int getWins() const { return wins; }

    string getName() const { return name; }

    void display() const {
        cout << name << " | Pts: " << points << " | GD: " << goalDiff << " | Wins: " << wins <<
endl;
    }

    static void displayTotalTeams() {
        cout << "\nTotal Teams in Tournament: " << totalTeamsInTournament << endl;
    }
}
```

```

};

int Team::totalTeamsInTournament = 0;

Team findGroupWinner(Team group[], int size) {
    Team winner = group[0];
    for (int i = 1; i < size; i++) {
        if (group[i].getPoints() > winner.getPoints()) {
            winner = group[i];
        } else if (group[i].getPoints() == winner.getPoints()) {
            if (group[i].getGoalDiff() > winner.getGoalDiff()) {
                winner = group[i];
            } else if (group[i].getGoalDiff() == winner.getGoalDiff()) {
                if (group[i].getWins() > winner.getWins()) {
                    winner = group[i];
                }
            }
        }
    }
    return winner;
}

int main() {
    Team groupA[2], groupB[2];
    cout << "--- Group A Data ---\n";
    for(int i = 0; i < 2; i++) groupA[i].inputTeam();
    cout << "\n--- Group B Data ---\n";
    for(int i = 0; i < 2; i++) groupB[i].inputTeam();
    cout << "\n===== TOURNAMENT RESULTS =====\n";
    Team winnerA = findGroupWinner(groupA, 2);
    Team winnerB = findGroupWinner(groupB, 2);
    cout << "Group A Winner: "; winnerA.display();
    cout << "Group B Winner: "; winnerB.display();
}

```

```

Team finalTeams[2] = {winnerA, winnerB};
Team overallChamp = findGroupWinner(finalTeams, 2);
Team::displayTotalTeams();
cout << "OVERALL TOURNAMENT CHAMPION: " << overallChamp.getName() <<
"!\\n";

return 0;
}

```

OUTPUT:

Output

```

--- Group A Data ---
Enter Team Name, Wins, Draws, Losses, Goals Scored, Goals Conceded:
Tigers 2 1 0 5 2
Lions 2 1 0 4 2
Enter Team Name, Wins, Draws, Losses, Goals Scored, Goals Conceded:

--- Group B Data ---
Enter Team Name, Wins, Draws, Losses, Goals Scored, Goals Conceded:
Eagles 3 0 0 6 1
Hawks 1 0 2 2 4
Enter Team Name, Wins, Draws, Losses, Goals Scored, Goals Conceded:

===== TOURNAMENT RESULTS =====
Group A Winner: Tigers | Pts: 7 | GD: 3 | Wins: 2
Group B Winner: Eagles | Pts: 9 | GD: 5 | Wins: 3

Total Teams in Tournament: 4
OVERALL TOURNAMENT CHAMPION: Eagles!

=== Code Execution Successful ===

```